Assignment 4: Temperature data for Montreal

Apply new Python knowledge of lists and files to analyzing weather data. Introduction to plotting libraries.

1. 📝 **Worth:** 6%
2. 🗓️ **Due Date:** See LEA
3. 🕑 **Late submissions:** 10% penalty per late day.
4. ⚓ **Starter code:**
   - “Distributed Documents” on LEA
   - `assignment4-starter.zip` contains:
     - `mtl_airport_temperature.csv`
     - `csv_indices.py`
     - `temperatures.py`
5. 📩 **Submission:**
   - Please use the provided files **WITHOUT** renaming them.
   - Include all 3 files in a `.zip` for your submission.
   - Fill the docstring header at the top of the `temperatures.py` file with your name, section and date.

Temperature Data for Montreal

The weather information from Dorval Airport, and then Pierre Elliot Trudeau Airport (same place, different name) is recorded and available on-line from Environment Canada.

This information is stored in a file called `mtl_airport_temperature.csv`, which is included in this starter. You can open this in a program like Excel (Windows) or Numbers (macOS) (or even in PyCharm) to see the complete data.

The data includes, but is not limited to:

- Year
- Month
- Day
- Temperature in °C: minimum, maximum and mean.
- Snow/rainfall

We will be analyzing the data, looking for “extreme” temperatures, specifically looking for the number of days that are very hot or very cold.

Part 1 - Reading the data

Read the CSV data and extract only the required columns. Create lists of the relevant data: (see the course notes: reading files instructions for the ‘how to’)

- Extract the columns for year, min daily and max daily temperature. Use the lists already created in the starter file.
- The file `csv_indices.py` contain constants for each column of the CSV file. Use these to reference the correct column in your code.
- Warning: not all of the lines of the CSV file have *good* data, so you need to verify that the line has the required information before you process. To do so, verify the the value is not equal to the empty string, `""` :

```
line[TEMP] != ""
```

Part 2 - Functions

Complete these functions in the file `temperatures.py` . It might help to read ahead to see where these might come in handy.

```
# ===============================================================================
# pass in the name of the file you want to read, and read the temperatures
# and save in the appropriate lists.
# DO NOT hard code the filename into this function
# ===============================================================================
def read_airport_temperatures(filename: str):
    """read the data from the given filename, and populate the appropriate lists""
```

```
# ============================================================================
# in the list data_col_years, you have each year repeated approximately 365 times
# - create a list of years where each year appears only once
# ============================================================================
def get_list_of_unique_years() -> list:
    """from the list of all years (the years listed in the 'years' column in the C
    create a list of years, where each year is never repeated"""

# ============================================================================
# for a given year, and a value, count how many times the max temperature is
# above the value (if check_below = False), or how many times the min temperature
# is below the given value (if check_below=True)
# ============================================================================
def get_extreme_temperature_count(this_year, value, check_below) -> int:
    """Count how many times the temperature is above/below the given value in the
```

Testing your functions

It's always a good idea to test your functions to gain confidence in your final result.

In the file `temperature.py`, you'll notice a funny `if`-statement:

```
if __name__ == "__main__":
    # Write any verification or testing code here
    print("Code in the body of this if-statement will run if you click play on thi
```

This is a nice place to add code to check that your functions are working as they should (i.e. use `print` statements).

Part 3 - Plot the number of extreme temperature days

Once you're confident your functions work correctly, you can proceed by plotting the data. For this task, we will use the scientific library `matplotlib`.

Setting up `matplotlib`

See the course notes:

- Install `matplotlib` in your project: Installing external libraries like `matplotlib`
- Using `matplotlib` : Introduction to `matplotlib`

The `plot` function

In the function `plot` , Use `matplotlib` , make a bar plot showing:

Above the y-axis:

- for each year in the data set, the number of days in that year with a MAX temperature at or above `HIGH_THRESHOLD` .

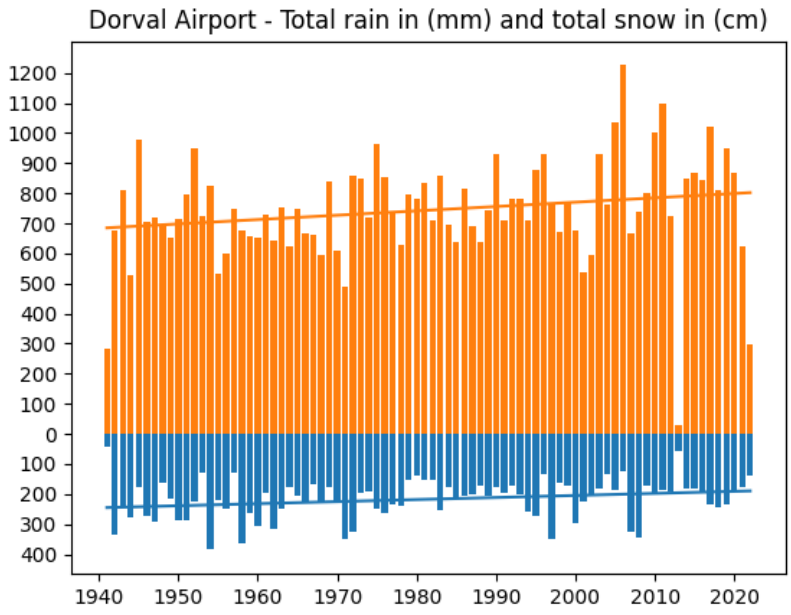
Below the y-axis:

- for each year in the data set, the number of days in that year with a MIN temperature at or below `LOW_THRESHOLD` .
- These positive values should be displayed in the *negative* direction. See the sample below.

Call the functions from Part 2 to produce this data.

Sample

Here is an example of how the bar plot should look, but with the temperature data of course!



Bar plot in `matplotlib`

Use the `bar` function to make a bar plot: (note, you can have more than one bar chart on a single plot)

```
plot.bar(years, data, color="RED")

plot.title('This is my title')
plot.xlabel('axis label')
plot.ylabel('axis label')
plot.show()
```

You can find more information at matplotlib.pyplot.bar.

Ticks

Since we are replacing the “negative” numbers the plot with positive numbers, you will need to configure the y-ticks (the labels on the y-axis). You can do this with the `yticks` function. For example,

```
plot.yticks([1,2,3], ["one", "two", "three"])
```

would replace the y-axis ticks with the words “one”, “two”, “three” at positions 1, 2, and 3, respectively. You can find more information at matplotlib.pyplot.yticks.

The required code has been given in the starter code.

Part 4 - Fitting the result

Add a straight-line fit for both sets of data: extreme hot and cold days.

Use the provided function `linear_fit` to get the slope and y-intercept of the straight line fit for your data. Here is an example of how to call this function:

```
m, b = linear_fit(x_data, y_data)
```

Output

Print the rate of change (# hot/cold days / year)

Part 5 - Code Quality

Ensure that all of your functions use Type hinting as covered in class.